Generating Invariants by Deductive Model Checking

Kyungmin Bae[1], Santiago Escobar[2], Raúl López-Rueda[2], José Meseguer[3], and Julia Sapiña[2]

[1] Pohang University of Science and Technology, Pohang, South Korea
[2] VRAIN, Universitat Politècnica de València, Valencia, Spain
[3] University of Illinois at Urbana-Champaign, Urbana, IL, USA.

Abstract. We present several significant new advances on the *deductive model checking* verification methodology for infinite-state systems supported by the DM-Check tool, which can prove a conjectured inductive invariant represented as a *disjunction of constrained patterns* by showing that the conjectured set is transition-closed. The new advances include: (i) several new inference rules to reason about semantic equivalence and generalization between constrained patterns; (ii) an *invariant generation* algorithm and methodology to compute the set of *all reachable states* from a disjunction of constrained patterns as a *fixpoint* representable as another disjunction of constrained patterns; (iii) a new version of the DM-Check tool supporting these new advances; and (iv) two case studies illustrating the practical usefulness of these advances.

1 Introduction

We present several significant advances in the *deductive model checking* (in the sense of [1,2]) of infinite-state concurrent systems specified as rewrite theories $\mathcal{R} = (\Sigma, E \cup B, R)$ [16]. The states of $\mathcal{R}$ are elements of the initial algebra $\mathbb{T}_{\Sigma/E\cup B}$, and its transitions are specified by the rewrite rules R. Infinite sets of states in $\mathcal{R}$ are represented as disjunctions of *constrained patterns* of the form: $u_1 \mid \varphi_1 \vee \ldots \vee u_n \mid \varphi_n$ where each $u_i \mid \varphi_i$ consists of a constructor term u_i denoting the (usually infinite) set of system states that are (ground) substitution instances of u_i, and a conjunction φ_i of Σ-equations that *constrain* the allowed ground substitutions ρ. That is, we only allow those ρ such that $\varphi\rho$ is inductively valid in $\mathbb{T}_{\Sigma/E\cup B}$. Such constraints (and therefore the sets of states they define) are very expressive, since the equational theory $(\Sigma, E \cup B)$ may contain arbitrarily complex recursive function definitions, which can be extended with other *auxiliary function definitions* added just to make constrained patterns even more expressive (see examples in Sections 4.1 and 5.2). This expressive power is really needed to specify important system properties such as invariants, but it comes at a cost. For example, the simple question of whether a constrained pattern $u \mid \varphi$ is *more general* (contains more states) than another one $v \mid \psi$ may require inductive theorem proving to be answered: we need to (i) find a B-matching substitution α such that $v =_B (u\alpha)$ (using a B-matching algorithm for axioms B such as

associativity and/or commutativity and/or unit element axioms); and then (ii) prove that $\psi \Rightarrow (\phi\alpha)$ is *valid* in $\mathbb{T}_{\Sigma/E\cup B}$ using an inductive theorem prover.

The key idea is to compute the set of states *reachable in one step* by transitions in R from the set of states denoted by $u_1 \mid \varphi_1 \vee \ldots \vee u_n \mid \varphi_n$ by performing *constrained narrowing* [14], which provides a *finite representation* of that set of states *reachable in one step* as another pattern disjunction $v_1 \mid \psi_1 \vee \ldots \vee v_m \mid \psi_m$. This provides a method to prove *inductive invariants* of $\mathcal{R}$, that is, sets of states that are *transition closed*. Indeed, $u_1 \mid \varphi_1 \vee \ldots \vee u_n \mid \varphi_n$ will be an inductive invariant iff the one-transition-step states denoted by $v_1 \mid \psi_1 \vee \ldots \vee v_m \mid \psi_m$ are a *subset* of the set of states denoted by $u_1 \mid \varphi_1 \vee \ldots \vee u_n \mid \varphi_n$. The problem is that, although the "model checking" reachability part is automatic by constrained narrowing, as we have already seen, just for two constrained patterns proving containment is nontrivial, since it requires inductive theorem proving. Proving containment between *disjunctions* of such patterns is even harder and *requires further deductive reasoning*, thus the "deductive" in "deductive model checking".

In [1,2] we presented a methodology for *proving inductive invariants* by deductive model checking using the DM-Check tool, which itself uses Maude's **NuITP** inductive theorem prover [8] as a backend. In this paper we report on the following significant new advances after [1,2]:

1. *Four new inference rules* that support deductive reasoning about the *semantic equivalence* between disjunctions of patterns. They complement other inference rules in [1,2] and are useful for proving containment between sets of states.
2. A *fifth new rule* to over-approximate disjunctions of constrained patterns by *generalization* (the inverse of containment).
3. A substantial *extension* of our deductive model checking methodology from the prior case of proving inductive invariants (which only requires a single step of constrained narrowing), to the more general case of *generating* inductive invariants by *several steps* of constrained narrowing. That is, we begin with a pattern disjunction $u_1 \mid \varphi_1 \vee \ldots \vee u_n \mid \varphi_n$ and try to obtain a *finite representation* of *all states* reachable from $u_1 \mid \varphi_1 \vee \ldots \vee u_n \mid \varphi_n$ (i.e., a *fixpoint* of the transition relation R) as a pattern disjunction $v_1 \mid \psi_1 \vee \ldots \vee v_m \mid \psi_m$.
4. A *new version* of the DM-Check tool that supports all of the new capabilities described in (1)–(3) above.
5. Two *case studies* demonstrating the practical usefulness of all these advances.

Related Work. We briefly discuss work closely related to DM-Check and refer to [2] for further details. The work closest to ours is [21] and the Maude Invariant Analyzer (InvA), an early approach that demonstrates the benefits of combining automated and interactive deduction; its functionality is subsumed by DM-Check. Invariant verification using proof scores in CafeOBJ [12,19,18,22] has also been applied to many case studies, but typically requires more manual effort than DM-Check. Our work is also related to narrowing-based reachability analysis and logical model checking [10,3,4,5,6,9], which usually rely on the finite variant property, thus restricting the class of systems that can be handled. Ivy [20,13] combines model checking with theorem proving for infinite-state systems, but emphasizes automatically discharging verification conditions using SMT.

2 Preliminaries

We assume familiarity with rewriting logic and refer the reader to [16,2].

We assume an order-sorted signature Σ and the term Σ-algebras $\mathbb{T}_\Sigma$ and $\mathbb{T}_\Sigma(X)$ for X an S-sorted set of variables. The subterm of term t at a term position p is denoted $t|_p$ and the term replacement of $t|_p$ by w at position p is denoted $t[w]_p$. Given an order-sorted substitution θ, its domain is $dom(\theta)$ and its range $ran(\theta)$. The application of a substitution θ to a term t is $t\theta$. The order-sorted equational deduction relation is denoted $E \vdash u = v$ and its associated E-equality relation $=_E$. The term Σ-algebras modulo E are $\mathbb{T}_{\Sigma/E}$ and $\mathbb{T}_{\Sigma/E}(X)$ and we denote the equivalence class of a term t by $[t]_E$ (or just $[t]$). The *E-unifiers* of an equation $u = v$ are substitutions θ such that $u\theta =_E v\theta$.

We assume an equational theory $(\Sigma, E \cup B)$ having a ground convergent decomposition $(\Sigma, B, \vec{E})$, a constructor subtheory $(\Omega, B_\Omega, \emptyset) \subseteq (\Sigma, B, \vec{E})$, and a sort *State* at the top of one of its connected components whose data elements denote states of a rewrite theory having $(\Sigma, B, \vec{E})$ as its equational subtheory. A constrained constructor pattern is of the form $u \mid \varphi$, where $u \in T_{\Omega,State}(X)$, with X_s countable for each sort s in Σ, and φ is a conjunction of Σ-equalities. We denote its semantics as $[\![u \mid \varphi]\!] = \{[v] \in T_{\Omega/B_\Omega,State} \mid \exists \rho \in [X \to T_\Omega] \ s.t. \ [v] = [u\rho] \wedge E \cup B \models \varphi\rho\}$. Pattern subsumption is denoted by $u \mid \varphi \sqsubseteq_{B_\Omega} v \mid \psi$, which by definition holds iff there exists a B_Ω-matching substitution α such that: (i) $u =_{B_\Omega} (v\alpha)$, and (ii) $\mathbb{T}_{\Sigma/E\cup B} \models \varphi \Rightarrow (\psi\alpha)$.

A rewrite theory $\mathcal{R} = (\Sigma, E \cup B, R)$ is *topmost* iff it has a sort *State* such that: (i) if $f(t_1, \ldots, t_n)$ has sort *State*, none of the t_i, $1 \leqslant i \leqslant n$ nor any subterm of any such t_i has sort *State*, and (ii) for any rewrite rule $l \to r$ *if* φ in $\mathcal{R}$, l and r have sort *State*. Constrained narrowing between constrained constructor terms is defined as $u \mid \varphi \overset{\alpha}{\rightsquigarrow}_{R,B_\Omega} v \mid \psi$ that holds iff there exists a rule $l \to r$ *if* ϕ in R and a disjoint[4] B_Ω-unifier α of the equation $u = l$ such that $v \mid \psi = (r \mid \varphi \wedge \phi)\alpha$.

3 DM-Check Methodology and Earlier Versions

DM-Check[5] [1,2] provides a novel deductive model checking methodology in which narrowing-based logical model checking of symbolic states—specified as disjunctions of constrained patterns—is combined with inductive theorem proving to discharge inductive verification conditions that ensure useful symbolic state space reductions. In fact, constrained narrowing [14] is a form of *deductive model checking* by itself, where symbolic model checking and inductive theorem proving are combined. In particular:

1. The existence of a ground substitution ρ such that $E \cup B \vdash (\psi \wedge \psi_j)\beta\rho$ is equivalent to saying that $\mathbb{T}_{\Sigma/E\cup B} \models \exists(\psi \wedge \psi_j)\beta$, where $\exists(\psi \wedge \psi_j)\beta$ denotes the existential closure of $(\psi \wedge \psi_j)\beta$; an *inductive* satisfiability property.

[4] θ is a disjoint B_Ω-unifier of $u = l$ iff $vars(u) \cap vars(l) = \emptyset$. Unifier disjointness can always be ensured by renaming variables.

[5] DM-Check is publicly available at `https://safe-tools.dsic.upv.es/dmc`.

2. Constrained narrowing search from $\bigvee_{i\in I} u_i \mid \varphi_i$ to find an intersection with $\bigvee_{j\in J} v_j \mid \psi_j$ will usually generate an infinite *narrowing forest* that symbolically describes the set of all states reachable from $[\![\bigvee_{i\in I} u_i \mid \varphi_i]\!]$.
3. However, if using the subsumption relation $\sqsubseteq_{B_\Omega}$ we can *fold* such an infinite narrowing tree into a *finite* narrowing graph, then only a finite number of constrained patterns need to be examined in finite time.
4. Folding means that any constrained pattern $u \mid \varphi$ found at depth $k+1$ is subsumed by some other constrained pattern $v \mid \psi$ found at depth $j \leqslant k$, i.e., $u \mid \varphi \sqsubseteq_{B_\Omega} v \mid \psi$. But then inductive theorem proving is essential, since, as explained in Section 2, for some matching substitution α we need to prove $\mathbb{T}_{\Sigma/E\cup B} \models \varphi \Rightarrow (\psi\alpha)$.

A simple integration of DM-Check with an inductive theorem prover is to use the inductive theorem prover in a terminating, automated mode as an oracle. But more powerful combinations are possible: DM-Check also allows that inductive verification conditions not discharged automatically by the oracle are dealt with by commands that refine some constrained patterns by useful semantic equivalences.

NuITP [8] is a next-generation inductive theorem prover written in Maude that combines advanced symbolic techniques such as narrowing, equality predicates, variant unification, variant satisfiability, order-sorted congruence closure, ordered rewriting, and strategy-based rewriting. NuITP implements the inductive inference system of [15,7] to verify properties of equational theories $(\Sigma, E \cup B)$ specified in Maude, where Σ is an order-sorted signature, B is any combination of the associativity (A), commutativity (C) and identity (U) axioms, and E is a set of ground convergent, possibly conditional equations. DM-Check uses NuITP as an oracle for two purposes: (i) to check subsumptions $u \mid \varphi \sqsubseteq_{B_\Omega} v \mid \psi$ for the constructor subtheory (Ω, B_Ω), and (ii) to check whether $u \mid \varphi$ denotes an empty set of states because φ is inductively unsatisfiable.

Invariants in DM-Check can be proved in several ways. Sections 4 and 5 below provide *new* commands. Commands supported in earlier versions [1,2] include:

Checking inductive invariant. The command "`check ind-invariant` $\bigvee_{i\in I} u_i \mid \varphi_i$" performs one step of constrained narrowing to check that $\bigcup_{i\in I}[\![u_i \mid \varphi_i]\!]$ is *transition-closed*. Any resulting constrained patterns not subsumed by $\bigvee_{i\in I} u_i \mid \varphi_i$ are returned as proof obligations. A default[6] NuITP proof strategy σ is used as an oracle to prove subsumptions and discard patterns with unsatisfiable constraints.
Verifying other invariants positively. The command "`check` $\bigvee_{i\in I} u_i \mid \varphi_i$ `subsumed by` $\bigvee_{j\in J} v_j \mid \psi_j$" tries to prove $\bigcup_{i\in I}[\![u_i \mid \varphi_i]\!] \subseteq \bigcup_{j\in J}[\![v_j \mid \psi_j]\!]$ using NuITP as an oracle. If DM-Check can prove this containment automatically only for a subset $I_0 \subseteq I$, it returns the remaining patterns $u_i \mid \varphi_i$ (for $i \in I \backslash I_0$) as proof obligations to be discharged using NuITP.
Verifying other invariants negatively. The command "`intersect` $\bigvee_{i\in I} u_i \mid \varphi_i$ `with` $\bigvee_{j\in J} v_j \mid \psi_j$" tries to prove $\bigcup_{i\in I}[\![u_i \mid \varphi_i]\!] \cap \bigcup_{j\in J}[\![v_j \mid \psi_j]\!] = \varnothing$. It symbolically computes the intersection using disjoint B_Ω-unification, invoking

[6] At present, a fixed default strategy is provided. In a future version the strategy will be user-definable.

NuITP as an oracle to discard any constrained patterns in the resulting disjunction whose constraints are shown unsatisfiable.

Adding lemmas. The command "`add lemma` φ" lets the user add a supporting lemma, which is sent to NuITP to help automatically discharge verification conditions. Instead of explicitly adding lemmas, the user may delegate intermediate results to NuITP by turning pending proof goals into lemmas.

- The command "`empty` *<id>* `of` *<parent-id>* `.`" indicates that a constrained pattern has no possible *valid* instantiation.
- The command "`lemma` *<id>* `of` *<parent-id>* `.`" marks the constrained pattern as proved, and generates a lemma from its condition.
- The command "`contained` *<id>* `of` *<parent-id>* `.`" indicates that a constrained pattern is folded into the original invariant, and a lemma is automatically generated from the associated subsumption.

All three commands assume *id* identifies one of the remaining constrained patterns, which is a child generated by one narrowing step from the constrained pattern identified by *parent-id* belonging to the original inductive invariant.

Case splitting. The command "`case` *<id>* `of` *<parent-id>* `on` *<variable>* `with` *<generator-set>* `.`" instantiates a variable in the constrained pattern *id*, which is a child generated by one narrowing step from the constrained pattern *parent-id* in the original inductive invariant, where the *generator-set* is a constructor-based generator set for the sort of that variable; see [2] for further details.

4 Pattern Equivalence and Generalization Commands

We propose five new commands that extend the reasoning capabilities of DM-Check in two different ways: (i) four new commands provide new ways of *preserving semantic equivalence* between constrained pattern disjunctions; they complement similar commands described in Section 3; and (ii) a *constrained pattern generalization* command, that allows a user to generalize constrained patterns into more general ones. The new semantic-equivalence-preserving commands are:

Variant Unify. If a constrained pattern has the form $u \mid \varphi \wedge t = t'$ ($\wedge$ is associative-commutative with identity $\top$), where t, t' are terms in an equational subtheory that enjoys the finite variant property, then the equation $t = t'$ has a finite number of variant unifiers, say, $\theta_1, \ldots, \theta_k$, $k \geqslant 0$, and the pattern $u \mid \varphi \wedge t = t'$ is semantically equivalent to the pattern disjunction

$$u\theta_1 \mid \varphi\theta_1 \vee \ldots \vee u\theta_k \mid \varphi\theta_k$$

Narrowing. If a constrained pattern has the form $u \mid \varphi \wedge t[f(u_1, \ldots, u_n)]_p = t'$ ($=$ is commutative), where p is a term position in t, the subterm $f(u_1, \ldots, u_n)$ at p is such that the $u_1, \ldots, u_n$ are constructor terms, f is a free function defined by equations (the case for conditional equations is similar) $E_f = \{f(v_{i.1}, \ldots, v_{i.n}) = r_i\}_{i \in I}$ (with the $v_{i.1}, \ldots, v_{i.n}$ constructor terms), and $f(u_1, \ldots, u_n)$ can be narrowed with the oriented equations E_f (modulo the axioms B of the equational theory) to, say, $\{f(u_1, \ldots, u_n) \overset{\alpha_j}{\rightsquigarrow}_{E_f, B_\Omega} w_j\}_{1 \leqslant j \leqslant k}$, then the pattern $u \mid$

$\varphi \wedge t[f(u_1, \ldots, u_n)]_p = t'$ is semantically equivalent to the pattern disjunction:

$$(u \mid \varphi \wedge t[w_1]_p = t')\alpha_1 \vee \ldots \vee (u \mid \varphi \wedge t[w_k]_p = t')\alpha_k$$

Variable Abstraction. If a constrained pattern has the form $u \mid \varphi \wedge t[v]_p = t'$, with v a non-variable subterm, then, it is semantically equivalent to the following constrained pattern, where x is a *fresh* variable whose sort is that of v:

$$u \mid \varphi \wedge t[x]_p = t' \wedge x = v$$

Substitution. If a constrained pattern has the form $u \mid \varphi \wedge x = v$, where the variable x does not occur in v and the smallest sort of v is smaller or equal to the sort of x, then it is semantically equivalent to the constrained pattern:

$$(u \mid \varphi)\{x \mapsto v\}$$

Generalization. This command generalizes a constrained pattern $u \mid \varphi$ to a *more general one* $v \mid \psi$, that is, to a pattern that (modulo the axioms B_Ω) *subsumes* $u \mid \varphi$, i.e., $u \mid \varphi \sqsubseteq_{B_\Omega} v \mid \psi$. The use of the **generalize** command to *over-approximate* states will be illustrated only in Section 5.2.

4.1 A QLOCK Example

The use of the new commands preserving pattern equivalence can be illustrated by an example from [17], where a Maude specification of the `QLOCK` mutual exclusion protocol originally proposed by K. Futatsugi was used to motivate the need for these new commands in DM-Check before they were implemented. The following rewrite rules define the behavior of QLOCK:

```
rl [r2w] : < U i | W | C | Q > => < U | W i | C | Q ; i > .
rl [w2c] : < U | W i | C | i ; Q > => < U | W | C i | i ; Q > .
rl [c2r] : < U | W | C i | i ; Q > => < U i | W | C | Q > .
```

Processes in `QLOCK` are abstractly modeled as natural numbers. The states of `QLOCK` have the general form `< U | W | C | Q >`, where `U` are the processes in the multiset *reception* area, `W` the processes in the multiset *waiting* area, and `C` the processes in the multiset *critical section*, and where `Q` is a list of process names awaiting to enter (or in) the critical section. The meaning of the rules `[r2w]` (reception to waiting), `[w2c]` (waiting to critical) and `[c2r]` (critical to reception) is self-explanatory and formalizes the protocol's semantics.

For a full specification of `QLOCK` and the proof of its properties see `https://safe-tools.dsic.upv.es/dmc/examples/qlock.html`. `QLOCK`'s specification also defines two predicates of sort `MSet -> Boolean` (no duplicates and nonempty) and a function of sort `List -> MSet` (mapping a list to its associated multiset):

```
eq set(mt) = tt .        eq set(i) = tt .        eq set(S S U) = ff .
ceq set(i U) = tt if i in U = ff /\ set(U) = tt .
eq non-mt(mt) = ff [variant] .                   eq l2ms(nil) = mt .
eq non-mt(S) = tt [variant] .                    eq l2ms(n ; L) = n l2ms(L) .
```

Note that only function `non-mt` has the `variant` equational attribute that specifies that these equations satisfy the *finite variant property* [11].

The new **DM-Check** commands are useful here to prove several properties of the `QLOCK` example. Note that `QLOCK` is *parametric* on the *infinite* set of initial states specified by the following constrained pattern, with `S` of sort `NeMSet`:

$$\texttt{< S | mt | mt | nil > | set(S) = tt} \tag{1}$$

The *inductive invariant* associated to `QLOCK` (i.e., transition-closed invariant) from the initial state pattern (1) is the *mutual exclusion property* defined as follows:

$$\begin{array}{r|l} \texttt{< U | W | mt | Q >} & \texttt{non-mt(U W) = tt} \wedge \texttt{set(U W) = tt} \wedge \texttt{l2ms(Q) = W} \quad \vee \\ \texttt{< V | T | i | i ; Q' >} & \texttt{non-mt(V T i) = tt} \wedge \texttt{set(V T i) = tt} \wedge \\ & \texttt{l2ms(i ; Q') = T i} \end{array} \tag{2}$$

The conjectured inductive invariant (2) can be verified by **DM-Check** by checking that it folds in one step using its `check ind-invariant` command:

```
DM-Check> check ind-invariant \
    ((< U:MSet | W:MSet | mt | Q:List >) | (non-xmt(U:MSet W:MSet) = tt) \
       /\ (set(U:MSet W:MSet) = tt) /\ (l2ms(Q:List) = W:MSet)) \
 \/ ((< V:MSet | T:MSet | i:Nat | i:Nat ; Q':List >) | \
      (non-mt(V:MSet T:MSet i:Nat) = tt) /\ (set(V:MSet T:MSet i:Nat) = tt) \
       /\ (l2ms(i:Nat ; Q':List) = T:MSet i:Nat)) .

    Invariant satisfied.
```

The inductive invariant satisfies that all its states in their critical section component are either `mt` or a single process `i`.

However, we still need to check that our inductive invariant (2) is actually an invariant *from* the initial state pattern (1). This can be checked by using the `check subsumed by` command:

```
DM-Check> check ((< S:NeMSet | mt | mt | nil >) | (set(S:NeMSet) = tt)) \
    subsumed by ((< U:MSet | W:MSet | mt | Q:List >) | \
                  (non-mt(U:MSet W:MSet) = tt) \
                   /\ (set(U:MSet W:MSet) = tt) /\ (l2ms(Q:List) = W:MSet)) \
            \/  ((< V:MSet | T:MSet | i:Nat | i:Nat ; Q':List >) | \
                  (non-mt(V:MSet T:MSet i:Nat) = tt) \
                   /\ (set(V:MSet T:MSet i:Nat) = tt) \
                   /\ (l2ms(i:Nat ; Q':List) = T:MSet i:Nat)) .

    Subsumption satisfied.
```

Any mutex algorithm worth its salt should be deadlock-free. Let us show that our inductive invariant from (1) is *deadlock free*, i.e., that all its states are *enabled* to perform some *transition* in `QLOCK`. The set of all *transition-enabled states* has a very simple expression as the disjunction of lefthand side patterns:

```
        < U1:MSet n1:Nat | W1:MSet | C1:MSet | Q1:List > | true ∨
  < U2:MSet | W2:MSet n2:Nat | C2:MSet | n2:Nat ; Q2:List >| true ∨
   < U3:MSet | W3:MSet | C3:MSet n3:Nat | n3:Nat ; Q3:List > | true
```

Deadlock freedom can then be proved with the following subsumption command; however, the subsumption check fails:

```
DM-Check> check ((< U:MSet | W:MSet | mt | Q:List >) | \
                (non-mt(U:MSet W:MSet) = tt) \
                 /\ (set(U:MSet W:MSet) = tt) /\ (l2ms(Q:List) = W:MSet)) \
            \/  ((< V:MSet | T:MSet | i:Nat | i:Nat ; Q':List >) | \
                (non-mt(V:MSet T:MSet i:Nat) = tt) \
                 /\ (set(V:MSet T:MSet i:Nat) = tt) \
                 /\ (l2ms(i:Nat ; Q':List) = T:MSet i:Nat)) \
 subsumed by \
  ((< U1:MSet n1:Nat | W1:MSet | C1:MSet | Q1:List >) | true) \
  \/ ((< U2:MSet | W2:MSet n2:Nat | C2:MSet | n2:Nat ; Q2:List >) | true) \
  \/ ((< U3:MSet | W3:MSet | C3:MSet n3:Nat | n3:Nat ; Q3:List >) | true) .

    Constrained terms on the left that could not be subsumed:

Term 4:  < $17:MSet | $18:MSet | mt | $16:List >
Matching: no matching found
Constraint 4:  (non-mt($17:MSet $18:MSet) = tt) /\ (set($17:MSet $18:MSet)
               = tt)  /\ l2ms($16:List) = $18:MSet
```

for the first pattern of the two from (2) because it is *too general* to show that all its ground instances are *transition-enabled*.

Since `non-mt` enjoys the finite variant property, we can use the new ***variant unify*** command to unify away the equation `non-mt(U W) = tt` using Maude's `filtered variant unify` command:

```
DM-Check> vunify to 4 with non-mt($17:MSet $18:MSet) =? tt .

    Constrained terms on the left that could not be subsumed:

Term 5:  < $19:NeMSet | mt | mt | $20:List >
Matching: no matching found
Constraint 5:  (set($19:NeMSet) = tt) /\ l2ms($20:List) = mt

Term 6:  < mt | $21:NeMSet | mt | $22:List >
Matching: no matching found
Constraint 6:  (set($21:NeMSet) = tt) /\ l2ms($22:List) = $21:NeMSet

Term 7:  < $23:NeMSet | $24:NeMSet | mt | $25:List >
Matching: no matching found
Constraint 7: (set($23:NeMSet $24:NeMSet) = tt) /\ l2ms($25:List) = $24:NeMSet
```

The previous `Term 4` becomes now equivalent to those `Term 5`, `Term 6` and `Term 7`.

We can now use the ***case*** command (which was already available in previous versions of DM-Check) with $\{m{:}Nat, (k{:}Nat\ S{:}NeMSet)\}$ as generator set for sort *NeMSet* to make `Term 5` semantically equivalent to two instances that can then be folded. Indeed, **DM-Check** returns the previous remaining terms after performing the case analysis and checking whether they are folded.

```
DM-Check> case to 5 on $19:NeMSet with m:Nat ;; k:Nat S:NeMSet .

    Constrained terms on the left that could not be subsumed:

Term 6:  < mt | $21:NeMSet | mt | $22:List >
Matching: no matching found
Constraint 6:  (set($21:NeMSet) = tt) /\ l2ms($22:List) = $21:NeMSet

Term 7:  < $23:NeMSet | $24:NeMSet | mt | $25:List >
Matching: no matching found
Constraint 7: (set($23:NeMSet $24:NeMSet) = tt) /\ l2ms($25:List) = $24:NeMSet
```

We can also apply the ***case*** command to `Term 7`, leaving only `Term 6`.

```
DM-Check> case to 7 on $23:NeMSet with m:Nat ;; k:Nat S:NeMSet .

    Constrained terms on the left that could not be subsumed:

Term 6:  < mt | $21:NeMSet | mt | $22:List >
Matching: no matching found
Constraint 6:  (set($21:NeMSet) = tt) /\ l2ms($22:List) = $21:NeMSet
```

Now, we can apply the new ***narrowing*** command to the term `l2ms($22:List)` in the constraint, which in this way becomes equivalent to a pattern disjunction:

```
DM-Check> narrow to 6 on l2ms($22:List) .

    Constrained terms on the left that could not be subsumed:

Term 7:  < mt | $23:NeMSet | mt | nil >
Matching: no matching found
Constraint 7:  (set($23:NeMSet) = tt) /\ mt = $23:NeMSet

Term 8:  < mt | $24:NeMSet | mt | $25:Nat ; $26:List >
Matching: no matching found
Constraint 8: (set($24:NeMSet) = tt) /\ ($25:Nat l2ms($26:List)) = $24:NeMSet
```

We are now only left with two patterns that can easily be discarded. First, by using the previous ***variant unify*** command:

```
DM-Check> vunify to 7 with $23:NeMSet =? mt .

    Constrained terms on the left that could not be subsumed:

Term 8:  < mt | $24:NeMSet | mt | $25:Nat ; $26:List >
```

```
Matching: no matching found
Constraint 8: (set($24:NeMSet) = tt) /\ ($25:Nat l2ms($26:List)) = $24:NeMSet
```

`Term 8` cannot be folded but using the new ***variable abstraction*** command on `l2ms(L:List)` with fresh variable `U:MSet` becomes semantically equivalent:

```
DM-Check> va to 8 on l2ms($26:List) with $27:MSet .

    Constrained terms on the left that could not be subsumed:

Term 9:  < mt | $27:NeMSet | mt | $28:Nat ; $29:List >
Matching: no matching found
Constraint 9:  (tt = set($27:NeMSet)) /\ ($30:MSet = l2ms($29:List)) /\
               $27:NeMSet = $28:Nat $30:MSet
```

If the term to be abstracted has several occurrences, this command will display the different available alternatives, allowing the user to select one by using the ***select va*** command. In the present case, however, only one abstraction is possible, and therefore ***va*** is applied automatically.

Finally, the last remaining pattern is discarded using the newly introduced ***substitution*** command, which finishes the proof of deadlock freedom for `QLOCK`:

```
DM-Check> substitution to 9 on $27:NeMSet with $28:Nat $30:MSet .

    Subsumption satisfied.
```

5 Generating Invariants by Deductive Model Checking

The new features of DM-Check, including generalization, make it possible to interactively generate an inductive invariant by deductive model checking. We present our methodology for generating invariants using DM-Check and illustrate it through a case study of Lamport's bakery protocol.

5.1 Fixpoint Generation

Our methodology is based on the folding narrowing search algorithm [17], which is adapted from narrowing-based model checking [3,10] and extended with constrained narrowing [14]. Given an initial pattern disjunction $u_1 \mid \varphi_1 \vee \cdots \vee u_n \mid \varphi_n$, the algorithm iteratively computes pattern disjunctions P_d and F_d—where F_d is called the *frontier* of P_d—for each depth $d \in \mathbb{N}$:

(1) $P_0 = F_0 = u_1 \mid \varphi_1 \vee \cdots \vee u_n \mid \varphi_n$.
(2) $P_{d+1} = P_d \vee F_{d+1}$, where F_{d+1} is the disjunction of the patterns obtained from F_d by one step of constrained narrowing and not subsumed by P_d.[7]

[7] Formally, $F_{d+1} = \{(w \mid \phi) \mid (\exists i)\ 1 \leqslant i \leqslant k, (u'_i \mid \varphi'_i \rightsquigarrow_{R,B_\Omega} w \mid \phi) \wedge (w \mid \phi) \not\sqsubseteq^{\sigma}_{B_\Omega} P_d\}$, where $F_d = \bigvee_{i=1,\ldots,k} u'_i \mid \varphi'_i$ and σ is a terminating subsumption proof strategy [17].

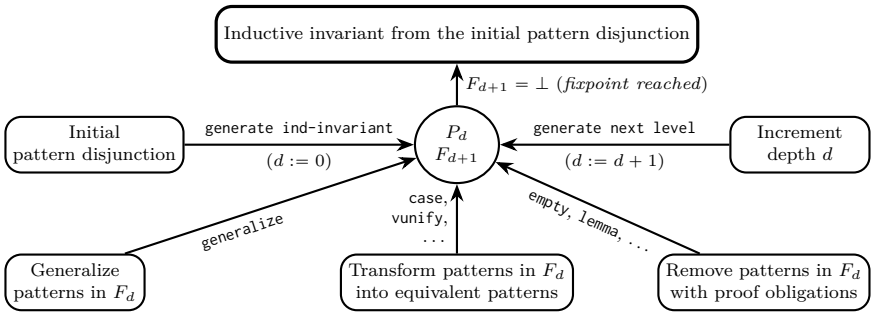

Fig. 1. Invariant generation by deductive model checking in DM-Check.

The algorithm finds an inductive invariant when it reaches a *fixpoint*. By construction, F_{d+1} *excludes* all patterns subsumed by P_d. Hence, if for some depth d every pattern obtained from F_d by constrained narrowing is subsumed by P_d, then $F_{d+1} = \emptyset$ and thus the algorithm reaches a fixpoint (i.e., $P_{d+1} = P_d$). In this case, P_d is an inductive invariant from the initial pattern [17], and captures all states reachable from the initial pattern $u_1 \mid \varphi_1 \vee \cdots \vee u_n \mid \varphi_n$.

In practice, such a fully automatic algorithm may not work for several reasons: (i) the initial pattern disjunction *may not be general enough* to reach a fixpoint, for example by starting with a disjunction of *ground* patterns; (ii) the algorithm may fail to reach a fixpoint at finite depth (looping) because some increasingly more complex patterns are generated that cannot be subsumed by other patterns; (iii) such looping may take place even though a fixpoint could have been reached at finite depth if actual set-theoretic containment could have been checked; this can happen because subsumption checking is only a *sufficient* condition for set-theoretic containment, and is further limited by the use of a fixed, terminating theorem proving strategy σ for checking implications between constraints.

These challenges for reaching a fixpoint can be addressed by providing not only an *automated* mode (which may certainly succeed in some cases), but also an *interactive* mode of deductive model checking depicted in Figure 1, where automated reachability analysis can be combined with deductive methods to: (i) *generalizing* patterns for over-approximation, (ii) proving *semantic equivalence* between pattern disjunctions; and (iii) proving *inductive lemmas* about implications between constraints and about constraint unsatisfiability.

To support methods (i)–(iii) in an interactive mode of deductive model checking we integrate the folding narrowing search mechanism with interactive commands in DM-Check for reasoning about pattern disjunctions. The commands `generate ind-invariant` and `generate next level` control the overall search, while the commands explained in Sections 3 and 4 support pattern generalization, pattern transformation, and proof obligation generation. Finally, NuITP can be used to discharge any remaining proof obligations.

5.2 Example: Lamport's Bakery Protocol

This section illustrates our methodology for generating invariants by deductive model checking, using a Maude specification of Lamport's bakery protocol [10,3]. In our previous work [2], DM-Check is used to verify the mutual exclusion of the bakery protocol for an unbounded number of processes; however, the full invariant is provided manually. In contrast, this section shows how an inductive invariant can be obtained interactively using the new features of DM-Check.

In the bakery protocol, multiple processes (or customers) compete to enter the critical section. Each process receives a ticket number to indicate that it is waiting, and the process with the smallest ticket number is served. Note that there are two sources of infiniteness: (i) due to the unbounded number of processes, the number of initial states is infinite; and (ii) due to unbounded ticket numbers, for each initial state, the number of reachable states is infinite.

Rewriting Logic Specification. A process is represented as a term of the form $[md]$, where its mode md is *idle* (not yet picked a number), $wait(j)$ (waiting with number j), or $crit(j)$ (being served with number j). A top-level state has the form $n \mid m \mid [md_1]\,;\cdots;[md_k]$, where n is the current number in the bakery's number dispenser, m is the number currently being served, and $[md_1]\,;\cdots;[md_k]$ is a semicolon-separated set of customer processes.

A set of processes is represented as a term of sort `ProcSet`—with two subsorts, `ProcIdleSet`, representing a set of idle processes, and `ProcWaitSet`, representing a set of idle or waiting processes—using an associative-commutative constructor `_;_` with identity element `none`. Similarly, natural numbers are represented as terms of sort `iNat`—with subsort `iNzNat` for non-zero natural numbers—using an associative-commutative constructor `_+_` with identity element 0.

We also define additional constructors and functions for specifying invariant properties. A set of ticket numbers is represented as a term of sort `MSet`, where an element with number n has the form `e(`n`)` and the empty set is `null`. The function `ticket(`*procs*`)` returns the set of ticket numbers occurring in a given set of processes *procs*. The function $i(n,m)$ returns the set of natural numbers $\{\, k \mid n \leqslant k < m \,\}$. For full details about the specification of `Bakery` see `https://safe-tools.dsic.upv.es/dmc/examples/bakery.html`.

The following rewrite rules define the behavior of the bakery protocol, where `PS` is a variable of sort `ProcSet`, and `N` and `M` are variables of sort `iNat`:

```
rl [wake] : N | M | [idle]    ; PS  =>  1 + N |     M | [wait(N)] ; PS .
rl [crit] : N | M | [wait(M)] ; PS  =>      N |     M | [crit(M)] ; PS .
rl [exit] : N | M | [crit(M)] ; PS  =>      N | 1 + M | [idle]    ; PS .
```

Initially, the two counters are equal and all processes are idle, as captured by the constrained pattern `(N | N | IS) | true`, where `IS` is a variable of sort `ProcIdleSet`. The error states violating the mutual exclusion property are described by the following constrained pattern, where `K` and `L` are variables of sort `iNat`: `(M | N | [crit(K)] ; [crit(L)] ; PS) | true`.

Interactive Deductive Model Checking. We begin by symbolically exploring the state space from the initial pattern using the `generate ind-invariant` command. As expected, the initial pattern is not inductive and DM-Check reports a successor pattern (some parts of the output are omitted and replaced with ...):

```
DM-Check> generate ind-invariant (N:iNat | N:iNat | IS:ProcIdleSet) | true .

    Invariant could not be proved.

Parent 1
Term:  $2:iNat | $2:iNat | $1:ProcIdleSet              Constraint:  true

Child 2 (Parent 1)
Term:  (1 + $4:iNat) | $4:iNat | $3:ProcIdleSet ; [wait($4:iNat)]
...
Matching: no matching found                             Constraint:  true
```

We continue the symbolic exploration by running the `generate next level` command. The child nodes then become parent nodes, and new child nodes are generated by constrained narrowing:

```
DM-Check> generate next level .

    Invariant could not be proved.

Parent 1
Term:  $2:iNat | $2:iNat | $1:ProcIdleSet              Constraint:  true

Parent 2
Term:  (1 + $4:iNat) | $4:iNat | $3:ProcIdleSet ; [wait($4:iNat)]
Constraint:  true

Child 3 (Parent 2)
Term:  (1 + 1 + $5:iNat) | $5:iNat | $6:ProcIdleSet ;
       [wait($5:iNat)] ; [wait(1 + $5:iNat)]            ...
Matching: no matching found                             Constraint:  true

Child 4 (Parent 2)
Term:  (1 + $7:iNat) | $7:iNat | $8:ProcIdleSet ; [crit($7:iNat)]
...
Matching: no matching found                             Constraint:  true
```

Further executions of this command do not lead to a fixpoint, but produce many patterns similar to Child 3, containing waiting processes with ticket numbers greater than or equal to the second counter but less than the first counter.[8] Based on this observation, we generalize Child 3 using the `generalize` command as follows (the output is the same as above except for Child 3):

[8] Typically, this command is executed multiple times to generate a variety of constrained patterns, allowing the user to identify generic patterns. Due to space limitations, we only show the result of a single execution of `generate next level`.

```
DM-Check> generalize to 3 using \
      (M:iNat + N:iNat | N:iNat | WS:ProcWaitSet) | \
             (tickets(WS:ProcWaitSet) = i(N:iNat, M:iNat + N:iNat)) .

    Invariant could not be proved.
...

Child 3 (Parent 2)
Term:  ($9:iNat + $10:iNat) | $10:iNat | $11:ProcWaitSet
...
Matching: no matching found
Constraint:  tickets($11:ProcWaitSet) = i($10:iNat, $9:iNat + $10:iNat)

...
```

A couple of executions of the `generate next level` command yield the following output. Only two parent nodes remain: all other parent nodes are subsumed by these nodes and therefore omitted from the output.

```
DM-Check> generate next level .
...
DM-Check> generate next level .

    Invariant could not be proved.

Parent 1
Term:  ($3:iNat + $1:iNat) | $1:iNat | $2:ProcWaitSet
Constraint:  tickets($2:ProcWaitSet) = i($1:iNat, $3:iNat + $1:iNat)

Parent 2
Term:  ($4:iNat + $5:iNat) | $4:iNat | $6:ProcWaitSet ; [crit($4:iNat)]
Constraint:  tickets($6 ; [wait($4)]) = i($4, $5 + $4)

Child 4 (Parent 2)
Term:  ($10:iNat + $11:iNat) | 1 + $10:iNat | $12:ProcWaitSet ; [idle]
...
Matching: no matching found
Constraint:  tickets($12 ; [wait($10)]) = i($10, $11 + $10)

Child 5 (Parent 2)
Term:  ($13:iNat + $14:iNat) | $13:iNat | $15:ProcWaitSet ;
       [crit($13:iNat)] ; [crit($13:iNat)]              ...
Matching: no matching found
Constraint:  tickets(($15 ; [wait($13)]) ; [wait($13)]) = i($13, $14 + $13)
```

Two child nodes are not subsumed by the parent nodes. The constraint of Child 5 is unsatisfiable, since $\mathtt{i}(\$19, \$20 + \$19)$ cannot contain duplicate numbers. The pattern of Child 4 is not sufficiently instantiated; so we split on whether `$11:iNat` is zero or non-zero. We therefore run the following commands:

```
DM-Check> empty 5 .
...
DM-Check> case to 4 on $11:iNat with 0 ;; 1 ;; (1 + #N:iNzNat) .

    Invariant could not be proved.
...

Child 5 (Parent 2)
Term:  $13:iNat | 1 + $13:iNat | $14:ProcWaitSet ; [idle]
...             Matching: no matching found
Constraint:  (e($13:iNat), tickets($14:ProcWaitSet)) = null

Child 6 (Parent 2)
Term:  (1 + $15:iNat) | 1 + $15:iNat | $16:ProcWaitSet ; [idle]
...             Matching parent #1: 1           ...
NuITP implication #1 (open):    ...

Child 7 (Parent 2)
Term:  (1 + $18:iNat + $17:iNzNat) | 1 + $18:iNat | $19:ProcWaitSet ; [idle]
...             Matching parent #1: 1           ...
NuITP implication #1 (open):    ...
```

The `case` command generates three new child nodes. We can easily see that Child 5 has a false constraint, and Children 6 and 7 have valid open implications. Thus, we run the `empty` and `lemma` commands as follows.

```
DM-Check> empty 5 .
...
DM-Check> lemma 6 .
...
DM-Check> lemma 7 .

    The following Invariant is satisfied

  ((($3:iNat + $1:iNat) | $1:iNat | $2:ProcWaitSet) |
    (tickets($2) = i($1, $3 + $1))) \/
  ((($4:iNat + $5:iNat) | $4:iNat | $6:ProcWaitSet ; [crit($4:iNat)]) |
    (tickets($6 ; [wait($4)]) = i($4, $5 + $4)))

    with the following proof obligations:.

  (e($10:iNat), tickets($12:ProcWaitSet)) = e($10:iNat + #N:iNzNat),
    i($10:iNat, $10:iNat + #N:iNzNat) -> tickets($12:ProcWaitSet ; [idle]) =
    i(1 + $10:iNat, #N:iNzNat + 1 + $10:iNat)
  (e($10:iNat), tickets($12:ProcWaitSet)) = e($10:iNat) ->
    tickets($12:ProcWaitSet ; [idle]) = i(1 + $10:iNat, 0 + 1 + $10:iNat)
  (e($19:iNat), tickets($20:ProcWaitSet)) = null -> false
  tickets(($14:ProcWaitSet ; [wait($15:iNat)]) ; [wait($13:iNat)]) =
    i($13:iNat, $14:iNat + $15:iNat) -> false
```

The output of the last command indicates that an inductive invariant is successfully generated, with four proof obligations remaining. All of the proof obligations can be discharged using NuITP (see `https://safe-tools.dsic.upv.es/dmc/examples/bakery.html`).

Proving Other Invariants. We can use the generated invariant to prove other invariants from the initial pattern. For example, the following command shows that, in each reachable state, the ticket numbers range from the second counter to the first counter (including the second but excluding the first):

```
DM-Check> check ($3:iNat + $1:iNat | $1:iNat | $2:ProcWaitSet) | \
        tickets($2:ProcWaitSet) = i($1:iNat, $3:iNat + $1:iNat) \/ \
    ($4:iNat + $5:iNat | $4:iNat | $6:ProcWaitSet ; [crit($4:iNat)]) | \
        tickets($6:ProcWaitSet ; [wait($4:iNat)]) = \
        i($4:iNat, $5:iNat + $4:iNat) \
  subsumed by (M:iNat + N:iNat | N:iNat | PS:ProcSet) | \
        tickets(PS:ProcSet) = i(N:iNat, N:iNat + M:iNat) .

    Subsumption satisfied.
```

The following command verifies that the generated invariant does not intersect with the error states that violate the mutual exclusion property. This proves that all states reachable from the initial pattern satisfy the mutual exclusion property, since the generated invariant includes all such reachable states.

```
DM-Check> intersect ($3:iNat + $1:iNat | $1:iNat | $2:ProcWaitSet) | \
        tickets($2:ProcWaitSet) = i($1:iNat, $3:iNat + $1:iNat) \/ \
    ($4:iNat + $5:iNat | $4:iNat | $6:ProcWaitSet ; [crit($4:iNat)]) | \
        tickets($6:ProcWaitSet ; [wait($4:iNat)]) = \
        i($4:iNat, $5:iNat + $4:iNat) \
  with \
    (M:iNat | N:iNat | [crit(K:iNat)] ; [crit(L:iNat)] ; PS:ProcSet) | true .

    No intersection.
```

6 Conclusions and Future Work

Up to now the deductive model checking methodology in [1,2] and its associated DM-Check tool were only applied to proving inductive invariants. In model checking terms, this amounts to the special case of *bounded* model checking with depth bound 1. Even for this subcase, the inference rules available for proving semantic equivalences between disjunctions of constrained patterns were limited, so that examples such as QLOCK could not be verified. In this work, this deductive model checking methodology has been significantly extended by: (i) new inference rules for proving such semantic equivalences; (ii) a generalization command for over-approximating sets of states; (iii) an *unbounded* deductive model checking

methodology to generate inductive invariants; and (iv) a new version of DM-Check supporting (i)—(iii). The usefulness of this extended methodology has been illustrated by means of the QLOCK and BAKERY examples.

Much work remains ahead. Firstly, further experimentation with other case studies and improvements to DM-Check should be pursued to make it more scalable. Secondly, *further automation* of this deductive model checking methodology should be investigated; for example by: (i) using more powerful terminating strategies in the **NuITP** to significantly increase the number of constrained patterns that can be automatically folded into previous ones; and (ii) applying some inference rules of DM-Check *automatically* by means of a *strategy*, e.g., a strategy could apply the `generalize` rule when some "looping" of constrained patterns is detected.

Acknowledgements S. Escobar, R. López-Rueda and J. Sapiña have been supported by the grant CIPROM/2022/6 funded by Generalitat Valenciana, and by the grant PID2024-162030OB-100 funded by MCIN/AEI/10.13039/501100011033 and ERDF A way of making Europe. K. Bae, S. Escobar, J. Sapiña, and R. López-Rueda have been supported by the NATO Science for Peace and Security Programme project SymSafe (grant number G6133). K. Bae was supported by the National Research Foundation of Korea (NRF) grant (No. RS-2021-NR060080) and the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant (No. RS-2024-00439856), both funded by the Korea government (MSIT).

References

1. Bae, K., Escobar, S., López-Rueda, R., Meseguer, J., Sapiña, J.: Verifying invariants by deductive model checking. In: Proceedings of the 15th International Workshop on Rewriting Logic and its Applications (WRLA 2024). Lecture Notes in Computer Science, vol. 14953, pp. 3–21. Springer (2024). https://doi.org/10.1007/978-3-031-65941-6_1
2. Bae, K., Escobar, S., López-Rueda, R., Meseguer, J., Sapiña, J.: DM-Check: Verifying invariants of concurrent systems by deductive model checking. Journal of Logical and Algebraic Methods in Programming **149**, 101107 (2026). https://doi.org/10.1016/j.jlamp.2025.101107
3. Bae, K., Escobar, S., Meseguer, J.: Abstract logical model checking of infinite-state systems using narrowing. In: Proceedings of the 24th International Conference on Rewriting Techniques and Applications (RTA 2013). Leibniz International Proceedings in Informatics (LIPIcs), vol. 21, pp. 81–96. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2013). https://doi.org/10.4230/LIPIcs.RTA.2013.81
4. Bae, K., Meseguer, J.: Infinite-state model checking of LTLR formulas using narrowing. In: Proceedings of the 10th International Workshop on Rewriting Logic and its Applications (WRLA 2014). Lecture Notes in Computer Science, vol. 8663, pp. 113–129. Springer (2014). https://doi.org/10.1007/978-3-319-12904-4_6
5. Bae, K., Meseguer, J.: Predicate abstraction of rewrite theories. In: RTA-TLCA. Lecture Notes in Computer Science, vol. 8560, pp. 61–76. Springer (2014)

6. Durán, F., Eker, S., Escobar, S., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.: Equational unification and matching, and symbolic reachability analysis in Maude 3.2 (system description). In: Proceedings of the 11th International Joint Conference on Automated Reasoning (IJCAR 2022). Lecture Notes in Computer Science, vol. 13385, pp. 529–540. Springer (2022). https://doi.org/10.1007/978-3-031-10769-6_31
7. Durán, F., Escobar, S., Meseguer, J., Sapiña, J.: NuITP alpha 30 – an inductive theorem prover for Maude equational theories (May 2024), available at https://nuitp.webs.upv.es/
8. Durán, F.J., Escobar, S., Meseguer, J., Sapiña, J.: NuITP: An inductive theorem prover for equational program verification. In: Proceedings of the 26th International Symposium on Principles and Practice of Declarative Programming, PPDP 2024, Milano, Italy, September 9-11, 2024. pp. 6:1–6:11. Association for Computing Machinery (2024). https://doi.org/10.1145/3678232.3678236
9. Escobar, S., López-Rueda, R., Sapiña, J.: Symbolic analysis by using folding narrowing with irreducibility and SMT constraints. In: Proceedings of the 9th International Workshop for Safety-Critical Systems (FTSCS 2023). pp. 14–25. Association for Computing Machinery (2023). https://doi.org/10.1145/3623503.3623537
10. Escobar, S., Meseguer, J.: Symbolic model checking of infinite-state systems using narrowing. In: Proceedings of the 18th International Conference on Term Rewriting and Applications (RTA 2007). Lecture Notes in Computer Science, vol. 4533, pp. 153–168. Springer (2007). https://doi.org/10.1007/978-3-540-73449-9_13
11. Escobar, S., Sasse, R., Meseguer, J.: Folding variant narrowing and optimal variant termination. The Journal of Logic and Algebraic Programming **81**(7–8), 898–928 (2012). https://doi.org/10.1016/j.jlap.2012.01.002
12. Futatsugi, K.: Advances of proof scores in CafeOBJ. Science of Computer Programming **224**, 102893 (2022). https://doi.org/10.1016/j.scico.2022.102893
13. McMillan, K.L., Padon, O.: Ivy: A multi-modal verification tool for distributed algorithms. In: Computer Aided Verification. pp. 190–202. Springer LNCS 12225 (2020)
14. Meseguer, J.: Generalized rewrite theories, coherence completion, and symbolic methods. Journal of Logical and Algebraic Methods in Programming **110** (2020). https://doi.org/10.1016/j.jlamp.2019.100483
15. Meseguer, J.: Inductive reasoning with equality predicates, contextual rewriting and variant-based simplification. Journal of Logical and Algebraic Methods in Programming **144**, 101036 (2025). https://doi.org/10.1016/J.JLAMP.2025.101036
16. Meseguer, J.: Conditional rewriting logic as a unified model of concurrency. Theoretical Computer Science **96**(1), 73–155 (1992)
17. Meseguer, J.: Symbolic computation and verification methods in maude. In: Escobar, S., Titolo, L. (eds.) Logic-Based Program Synthesis and Transformation - 35th International Symposium, LOPSTR 2025, Rende, Italy, September 9-10, 2025, Proceedings. Lecture Notes in Computer Science, vol. 16117, pp. 1–21. Springer (2025). https://doi.org/10.1007/978-3-032-04848-6_1, https://doi.org/10.1007/978-3-032-04848-6_1
18. Ogata, K., Futatsugi, K.: Proof scores in the OTS/CafeOBJ method. In: Formal Methods for Open Object-Based Distributed Systems, 6th IFIP WG 6.1 International Conference, FMOODS 2003, Paris, France, November 19.21, 2003, Proceedings. Lecture Notes in Computer Science, vol. 2884, pp. 170–184. Springer (2003). https://doi.org/10.1007/978-3-540-39958-2_12

19. Ogata, K., Futatsugi, K.: Theorem proving based on proof scores for rewrite theory specifications of OTSs. In: Specification, Algebra, and Software - Essays Dedicated to Kokichi Futatsugi. Lecture Notes in Computer Science, vol. 8373, pp. 630–656. Springer (2014)
20. Padon, O., McMillan, K.L., Panda, A., Sagiv, M., Shoham, S.: Ivy: safety verification by interactive generalization. In: Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation. pp. 614–630. PLDI '16, ACM (2016)
21. Rocha, C., Meseguer, J.: Mechanical analysis of reliable communication in the alternating bit protocol using the Maude invariant analyzer tool. In: Specification, Algebra, and Software - Essays Dedicated to Kokichi Futatsugi. Lecture Notes in Computer Science, vol. 8373, pp. 603–629. Springer (2014)
22. Tran, D.D., Ogata, K.: IPSG: Invariant proof score generator. In: 46th IEEE Annual Computers, Software, and Applications Conferenc, COMPSAC 2022, Los Alamitos, CA, USA, June 27 - July 1, 2022. pp. 1050–1055. IEEE (2022). https://doi.org/10.1109/COMPSAC54236.2022.00164